Chapter 3
Requirements Engineering Principles

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Requirement Engineering

- **Requirements**: specify what to build
  - tell "what" and not "how"
  - tell the system design, not the software design
  - tell the problem, not the solution (in detail)
- It may range from a high-level abstract statement of a service or of a system constraint to a detailed mathematical functional specification.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Requirements Engineering

User Requirements

- Often referred to as user needs
- Describes activities that users must be able to perform
- Includes services the system provides and its operational constraints
- User requirements are generally signed off by the user and used as the primary input for creating system requirements.
- User requirements are generally considered separately from system requirements.
- The business analyst carefully analyzes user requirements and carefully constructs and documents ensuring that the requirements meet certain quality characteristics.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Requirements Engineering

System Requirements

- System requirements are the building blocks developers use to build the system.
- The system requirements are description of features and functionalities of the target system.
- A structured document setting out detailed descriptions of the system's functions, services and operational constraints
- A structured document setting out detailed descriptions of the system services.
- It is written as a contract between client and contractor.

Available on notes.arpanadhikari7.com.np

Functional Requirements

Functional requirements specify the features, behaviors, and functionalities that a system or application must provide. These are the "what" of a system, describing what the software should do to meet user needs or business goals.

- Services the system should provide
- Describe functionality or system services.
- It describes the functions a software must perform.
- Statements of how the system should react to particular inputs and how the system should behave in particular situations.
- Functional software requirements help us to capture the intended behavior of the system.
- May state what the system should not do
- Functional Requirements depend on the type of software, expected users and the type of system where the software is used.
- Functional user requirements may be high-level statements of what the system should do
- Functional system requirements should describe the system services in detail.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

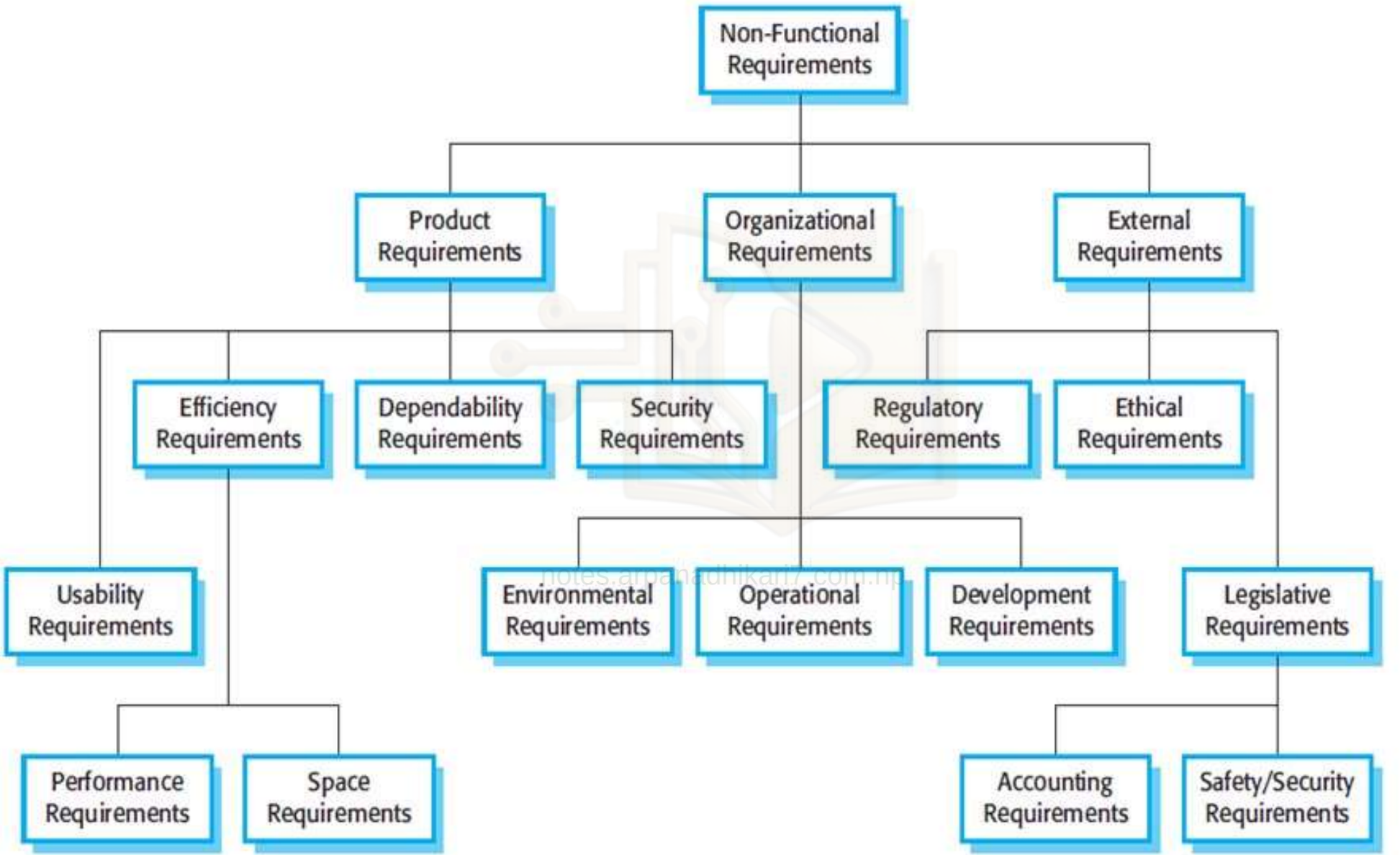
Non-Functional requirements

- Non-Functional Requirement (NFR) specifies the quality attribute of a software system.
- These define system properties and constraints
- Non-functional requirements are often called "quality attributes" of a system.
- Non-functional requirements generally specify the system’s quality attributes or characteristics.
- They judge the software system based on Responsiveness, Usability, Security, Portability and other non-functional standards that are critical to the success of the software system.
- Non-functional Requirements allows you to impose constraints or restrictions on the design of the system

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Non-Functional Requirement Types



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Non-Functional Requirement

➢Some Non-Functional Requirements are:

1. Availability: It is the time frame in which the system functions normally and without failures.
2. Efficiency: It is the system capability to management resources as CPU, Memory and Network Infrastructure
3. Performance: It is the system capability to do specific actions in a given time span.
4. Reliability: It is measured as the probability that the software will not fail.
5. Robustness: The ability of the system to control and continue to work with exceptions, unexpected conditions without affecting the user's experience.
6. Accuracy: System capacity to perform calculations or algorithms with a certain precision

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Non-Functional Requirement

7. Security: System capacity to avoid malicious incidents or activities. Its must keep in mind the Access, Privacy, Integrity and Audit of its information.
8. Interoperability: It is the system capability to communicate successfully with different external systems.
9. Maintainability: How fast the system can be maintained or changes can be made.
10. Extensibility: It is the system capability to easily add new modules in a planned way.
11. Manageability: It is the capability of the system to monitor and manage the system by administrators.
12. Portability: It is the ability of the system to run on different computing platforms.
13. Reusability: It is the capacity that defines the degree to which part of the system can be re-used.
14. Usability: Define the degree of ease with which the user can learn and operate

Available on notes.arpanadhikari7.com.np

Functional vs Non-Functional Requirements

Functional Requirements	Non-Functional requirements
Functional requirements are those requirements that deal with what the system should do or provide for the users.	Non-Functional Requirements(NFR) are those requirement that specifies criteria that can be used to judge the operation of a system, rather than specific behaviors.
Describe the action with which the user's work is connected.	Describe the experience of the user while doing the work.
These are the functions that can be captured on use case.	These are the global constraints on software system.
Functional Requirements are mandatory	Non-Functional Requirements are not mandatory.
These requirements are easy to capture.	Hard to capture.
Involves Functional Testing like System, Integration, End to End, API testing, etc.	Involves Non-Functional Testing like Performance, Stress, Usability, Security testing, etc.
Example: Consider a flight reservation system, where a user can search for a flight. If a flight is selected, the user has 2 minutes to confirm the booking or the flight will be released	Example: For the flight reservation system, the response time for a search transaction under a peak load of 10,000 users must not exceed 2 seconds

Available on notes.arpanadhikari7.com.np

Functional Requirements Example: ABC Contact Management System

1. The software automatically validates customers against the ABC Contact Management System
2. The Sales system should allow users to record customers sales
3. The background color for all windows in the application will be blue and have a hexadecimal RGB color value of 0x0000FF.
4. Only Managerial level employees have the right to view revenue data.
5. The software system should be integrated with banking API

notes.arpanadhikari7.com.np

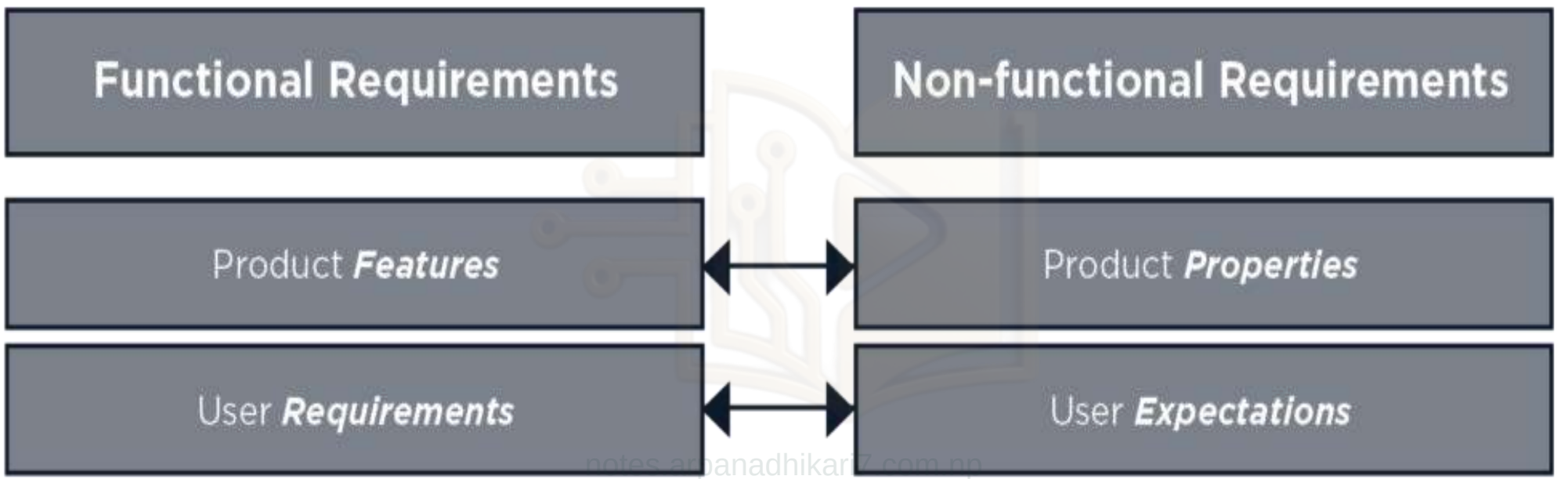
Available on notes.arpanadhikari7.com.np

Non-Functional Requirements Example: ABC Contact Management System

1. Employees never allowed to update their salary information. Such attempt should be reported to the security administrator.
2. Every unsuccessful attempt by a user to access an item of data shall be recorded on an audit trail.
3. A website should be capable enough to handle 20 million users with affecting its performance
4. The software should be portable. So moving from one OS to other OS does not create any problem.
5. Privacy of information, the export of restricted technologies, intellectual property rights, etc. should be audited.

Available on notes.arpanadhikari7.com.np

Functional and Non-Functional Requirements



notes.arpanadhikari7.com.np

Requirement Engineering

- Requirement Engineering is the process of defining, documenting and maintaining the requirements.
- Requirements engineering is a process of gathering and defining of what the services should be provided by the system.
- The goal of requirement engineering is to develop and maintain sophisticated and descriptive ‘System Requirements Specification’ document.
- Requirement engineering provides the appropriate mechanism to:
    1. understand what the customer desires
    2. analyzing the need, and assessing feasibility
    3. negotiating a reasonable solution
    4. specifying the solution clearly
    5. validating the specifications and managing the requirements

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Requirement Engineering Process Activities

1. Feasibility Study
2. Requirements Elicitation and Analysis
3. Requirements Validation
4. Requirements Management

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

1. Feasibility Study

- A feasibility study decides whether or not the proposed system is worthwhile.
- During Feasibility Study, it is studied whether the desired system and its functionality are feasible to develop.
- Feasibility study is focused towards goal of the organization.
- A short focused study that checks:
  - If the system contributes to organizational objectives
  - If the system can be engineered using current technology and within budget
  - If the system can be integrated with other systems that are used
- The output of this phase should be a feasibility study report that should contain adequate comments and recommendations for management about whether or not the project should be undertaken.

Available on notes.arpanadhikari7.com.np

Types of Feasibility Study

1. Technical Feasibility

- In Technical Feasibility current resources both hardware software along with required technology are analyzed/assessed to develop project.
- This technical feasibility study gives report whether there exists correct required resources and technologies which will be used for project development.
- Along with this, technical feasibility study also analyzes:

- Technical skills and capabilities of technical team
- Existing technology can be used or not
- Maintenance and up-gradation is easy or not for chosen technology etc.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Types of Feasibility Study

2. Economic Feasibility

- This assessment typically involves:
  - a cost/ benefits analysis of the project
  - helping organizations determine the viability, cost, and benefits associated with a project before financial resources are allocated.
- After that it is analyzed whether project will be beneficial in terms of finance for organization or not.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Types of Feasibility Study

3. Operational Feasibility

- This assessment involves undertaking a study to analyze and determine whether and how well the organization's needs can be met by completing the project.
- Degree of providing service to requirements is analyzed along with how much easy product will be to operate and maintain.
- Determines suggested solution by software development team is acceptable or not

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Types of Feasibility Study

4. Legal Feasibility

- In Legal Feasibility study, project is analyzed in legality point of view.
- This includes analyzing barriers of legal implementation of project, data protection acts or social media laws, project certificate, license, copyright etc.
- Overall it can be said that Legal Feasibility Study is study to know if proposed project conform legal and ethical requirements.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Types of Feasibility Study

5. Schedule Feasibility

- In Schedule Feasibility Study, mainly timelines/deadlines is analyzed for proposed project.
- It includes how many times teams will take to complete final project which has a great impact on the organization as purpose of project may fail if it can’t be completed on time.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Key benefits of conducting a Feasibility Study

- Improves project team's focus
- Identifies new opportunities
- Provides valuable information for a "go/no-go" decision
- Narrows the business alternatives
- Identifies a valid reason to undertake the project
- Enhances the success rate by evaluating multiple parameters
- Aids decision-making on the project
- Identifies reasons not to proceed

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

2. Requirement Elicitation and Analysis

- Requirements Elicitation is the practice of researching and discovering the requirements of a system.
- Requirement Elicitation is also sometimes referred to as "requirement gathering"
- Requirement Elicitation and Analysis is a process of interacting with customers and end-users to find out about the domain requirements, what services the system should provide, and the other constraints
- There are four main activities during requirement elicitation and analysis.

1. Requirements Discovery
2. Requirements Classification and Organization
3. Requirements Prioritization and Negotiation
4. Requirements Specification

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

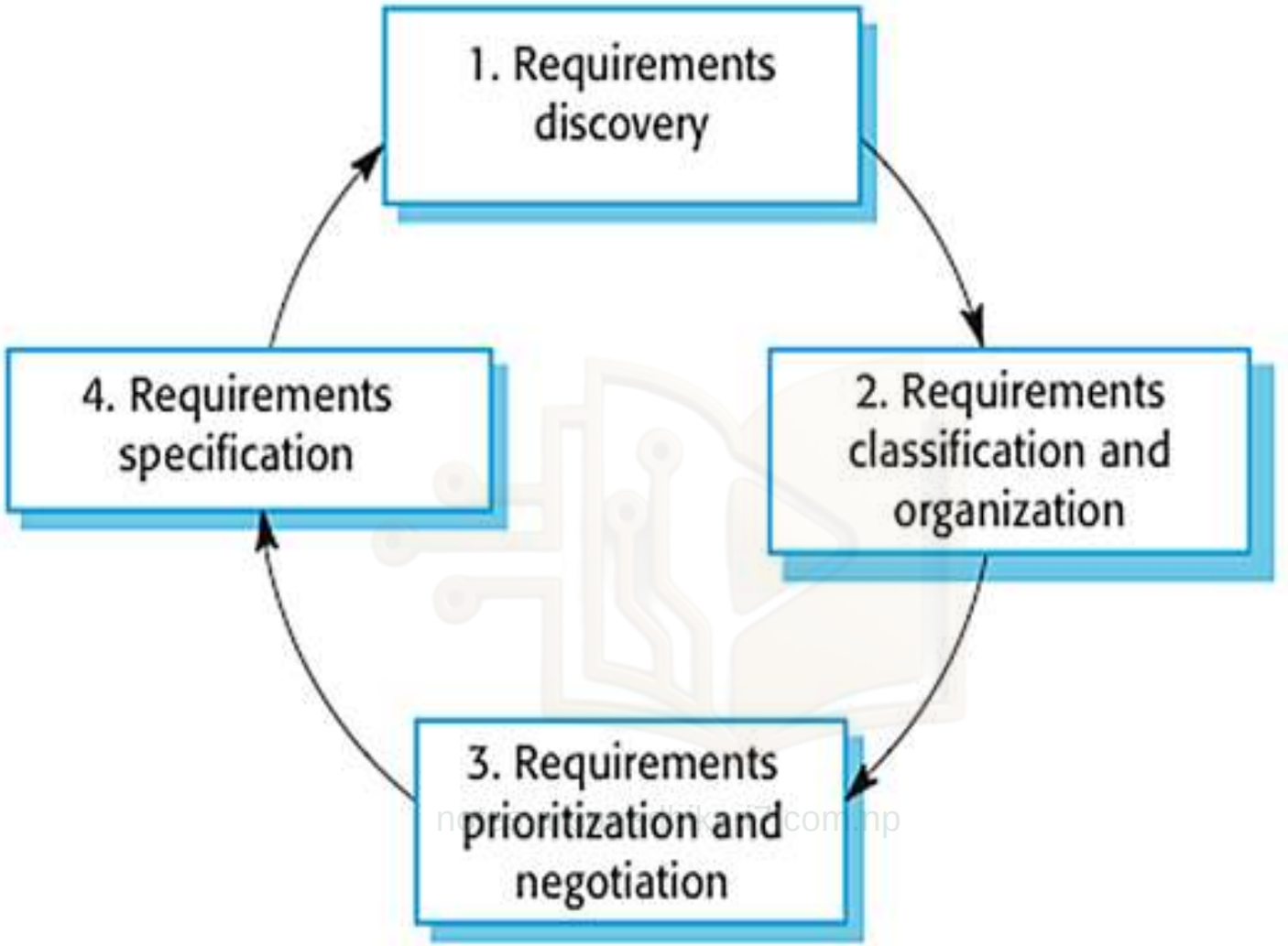


Figure: The process of requirements elicitation and analysis

Available on notes.arpanadhikari7.com.np

2.1. Requirements Discovery

- It's the process of interacting with, and gathering the requirements from, the stakeholders about the required system and the existing system
- Gathering and understanding the requirements is a difficult process
- It can be done using some techniques like:

a. Interviews
b. Brainstorming Sessions
c. Facilitated Application Specification Technique (FAST)
d. Quality Function Deployment (QFD)
e. Use Cases and Scenarios
f. Requirements reuse
g. Observations and social analysis

notes.arpanadhikari7.com.np

These techniques are also known as requirement elicitation techniques

Available on notes.arpanadhikari7.com.np

a. Interview

- Objective of conducting an interview is to understand the customer's expectations from the software.
- It is impossible to interview every stakeholder hence representatives from groups are selected based on their expertise and credibility.
- Interviews maybe be open ended or structured.

1. In open ended interviews there is no pre-set agenda. Context free questions may be asked to understand the problem.
2. In structured interview, agenda of fairly open questions is prepared. Sometimes a proper questionnaire is designed for the interview.

- In practice, interviews usually use mixture of both.
- Usually, start with the open-ended questions, and then use the closed-ended questions to be more specific about some requirements

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

b. Brainstorming Sessions

- It is a group technique
- It is intended to generate lots of new ideas hence providing a platform to share views
- A highly trained facilitator is required to handle group bias and group conflicts.
- Every idea is documented so that everyone can see it.
- Finally a document is prepared which consists of the list of requirements and their priority if possible.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

c. Facilitated Application Specification Technique(FAST)

- A team oriented approach is developed for needs gathering.
- This approach supports the creation of a joint team of customers and developers who work together to understand the expectations and propose of set of requirements.
- Each attendee is asked to make a list of objects that are-
  - Part of the environment that surrounds the system
  - Produced by the system
  - Used by the system
- Here each participant prepares his/her list, different lists are then combined, redundant entries are eliminated, team is divided into smaller sub-teams to develop mini-specifications and finally a draft of specifications is written down using all the inputs from the meeting.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

d. Quality Function Deployment

- In this technique customer satisfaction is of prime concern.
- It emphasizes on the requirements which are valuable to the customer.
- 3 types of requirements are identified:

- **Normal requirements:** In normal requirements, the objective and goals of the proposed software are discussed with the customer
- **Expected requirements:** These requirements are so obvious that the customer need not explicitly state them.
- **Exciting requirements:** These requirements include features that are beyond customer's expectations and prove to be very satisfying when present.

Available on notes.arpanadhikari7.com.np

d. Quality Function Deployment

➢The major steps involved in this procedure are:

1. Identify all the stakeholders, eg. Users, developers, customers etc.
2. List out all requirements from customer.
3. A value indicating degree of importance is assigned to each requirement.
4. In the end the final list of requirements is categorized as:
   - It is possible to achieve
   - It should be deferred and the reason for it
   - It is impossible to achieve and should be dropped off

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

e. Use Cases and Scenarios

- Use cases identify interactions between the system and it's users or even other external systems.
- A scenario is a textual description of one or more of these interactions.
- This technique combines text and picScenarios are descriptions of how a system is used in practice.
- helps to provide a better understanding of the requirements.
- The use cases describe the 'what', of a system and not 'how'. Hence they only give a functional view of the system.
- Use case and scenarios are effective techniques for eliciting the requirements.
- Since they focus on the interactions with the system, they aren't effective for eliciting high-level business, non-functional, or domain requirements.

Available on notes.arpanadhikari7.com.np

f. Requirements Reuse

- Reuse involves taking the requirements which have been developed for one system and using them in a different system.
- Requirements reuse saves time and effort as reused requirements have already been analyzed and validated in other systems

g. Observations and social analysis

- Requirements can be gathered from observing existing systems as well as via social analysis.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

2.2. Requirements Classification and Organization

- It is the second phase in requirements elicitation and analysis.
- The requirements gathered in first phase are classified and organized in this phase.
- It involves putting related requirements together, and decomposing the system into sub components of related requirements.
- Then, we define the relationship between these components.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

2.3 Requirements Prioritization and Negotiation

- This activity is concerned with prioritizing requirements and finding and resolving requirements conflicts through negotiations until we reach a situation where some of the stakeholders can compromise.
- We shouldn't reach a situation where a stakeholder is not satisfied because his requirements is not taken into consideration.
- Prioritizing requirements will help us later to focus on the essentials and core features of the system, so we can meet the user expectations.
- It can be achieved by giving every piece of function a priority level.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

2.4 Requirements Specification

- It’s the process of writing down the user and system requirements into a document.
- A detailed software description which can serve as a basis for a design or implementation.
- A Software Requirements Specification (SRS) is a detailed description of a software system to be developed with its functional and non-functional requirements.
- An SRS minimizes the time and effort required by developers to achieve desired goals and also minimizes the development cost.
- It may include the use cases of how user is going to interact with software system.
- The requirements should be clear, easy to understand, complete and consistent.
- In practice, this is difficult to achieve as stakeholders interpret the requirements in different ways and there are often inherent conflicts and inconsistencies in the requirements.

Available on notes.arpanadhikari7.com.np

System/Software Requirements Specification(SRS)

➢Qualities of Good SRS:

- Correct
- Unambiguous
- Complete
- Consistent
- Ranked for importance and/or stability
- Verifiable
- Modifiable
- Traceable

notes.arpanadhikari7.com.np

System/Software Requirements Specification(SRS)

Before implementing SRS following things need to be considered:

1. Correctness of SRS should be checked.
2. Ambiguity should be avoided
3. Requirements should be complete
4. Consistent requirements
5. Verification of expected result
6. Testing environment
7. Pre-conditions defined clearly
8. Security and Performance criteria
9. Assumption should be avoided
10. Deletion of irrelevant requirements

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

3. Requirements Validation

- Requirement Validation is the process of ensuring the specified requirements meet the customer needs.
- Requirements validation is the process of checking that requirements defined for development, define the system that the customer really wants.
- To check issues related to requirements, we perform requirements validation.
- We usually use requirements validation to check error at the initial phase of development as the error may increase excessive rework when detected later in the development process.
- In the requirements validation process, we perform a different type of test to check the requirements mentioned in the Software Requirements Specification (SRS)

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

3. Requirements Validation

➢These checks include:

1. Validity checks: The functions proposed by stakeholders should be aligned with what the system needs to perform. We may find later that there are additional or different functions are required instead.
2. Consistency checks: Requirements in the document shouldn't conflict or different description of the same function.
3. Completeness checks: The document should include all the requirements and constraints.
4. Realism checks: Ensure the requirements can actually be implemented using the knowledge of existing technology, the budget, schedule, etc.
5. Verifiability: Requirements should be written so that they can be tested. This means you should be able to write a set of tests that demonstrate that the system meets the specified requirements.
6. Ambiguity checks: Testing for ambiguity/ambiguous requirements

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Techniques used to Validate Requirements

1. **Requirements Reviews**
   - The SRS is carefully reviewed by a group of people including people from both the organization and the client side
   - The reviewer systematically analyses the document to check error and ambiguity.
2. **Prototyping**
   - In this approach to validation, an executable model of the system is demonstrated to the customer and end users to validate, and ensure if it meets their needs.
   - Prototyping is usually used when the requirements aren’t clear.
3. **Test case generation**
   - Requirement mentioned in SRS document should be testable, the conducted tests reveal the error present in the requirement.
   - It is generally believed that if the test is difficult or impossible to design than, this usually means that requirement will be difficult to implement and it should be reconsidered.

Available on notes.arpanadhikari7.com.np

4. Automated Consistency Analysis

- This approach is used for automatic detection of an error
- First, the requirement is structured in formal notation then CASE tool is used to check in-consistency of the system.
- The report of all inconsistencies is identified and corrective actions are taken.

5. Walk-through

- A walkthrough does not have a formally defined procedure and does not require a differentiated role assignment.
- Basically walkthrough here involves:

a. Checking early whether the idea is feasible or not.
b. Obtaining the opinions and suggestion of other people.
c. Checking the approval of others and reaching an agreement.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

4. Requirements Management

- Requirements management is the process of collecting, analyzing, refining, and prioritizing product requirements and then planning for their delivery.
- The purpose of requirements management is to ensure that the organization validates and meets the needs of its customers and external and internal stakeholders.
- The purpose of requirements management is to assure the organization documents verifies and meets the needs and expectations of its customers and internal or external stakeholders.
- The essential activities performed in requirements management are listed below.
  - Recognizing the need for change in the requirements
  - Establishing a relationship amongst stakeholders and involving them in the requirements engineering process
  - Identifying and tracking requirements attributes.

Available on notes.arpanadhikari7.com.np

Advantages of Requirement Management

1. Better control of complex projects
2. Reduced project costs and delays
3. Improved software quality
4. Improved team communication
5. Easing compliance with standards and regulations
6. Complete Specification of User requirements

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Requirements Change - Why requirements change?

- Requirements Change is one of the critical problems faced in software development

1. Poorly Defined Requirement Development Process
   - A major reason for change is a poorly defined or ignored requirement development process.
   - This can result in defective requirements, incorrect requirements, and missing requirements.
   - The further in the development life cycle we go before discovering these problems, the greater the impact on cost and schedule.
   - At this situation, developers find major problems and issues with the requirements and so the changes begin
2. Ignored Requirements
   - The developers could have ignored some of the original requirements.
   - Either they didn't understand the requirements or the requirements were too complex or the developer just chose to ignore the requirements.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

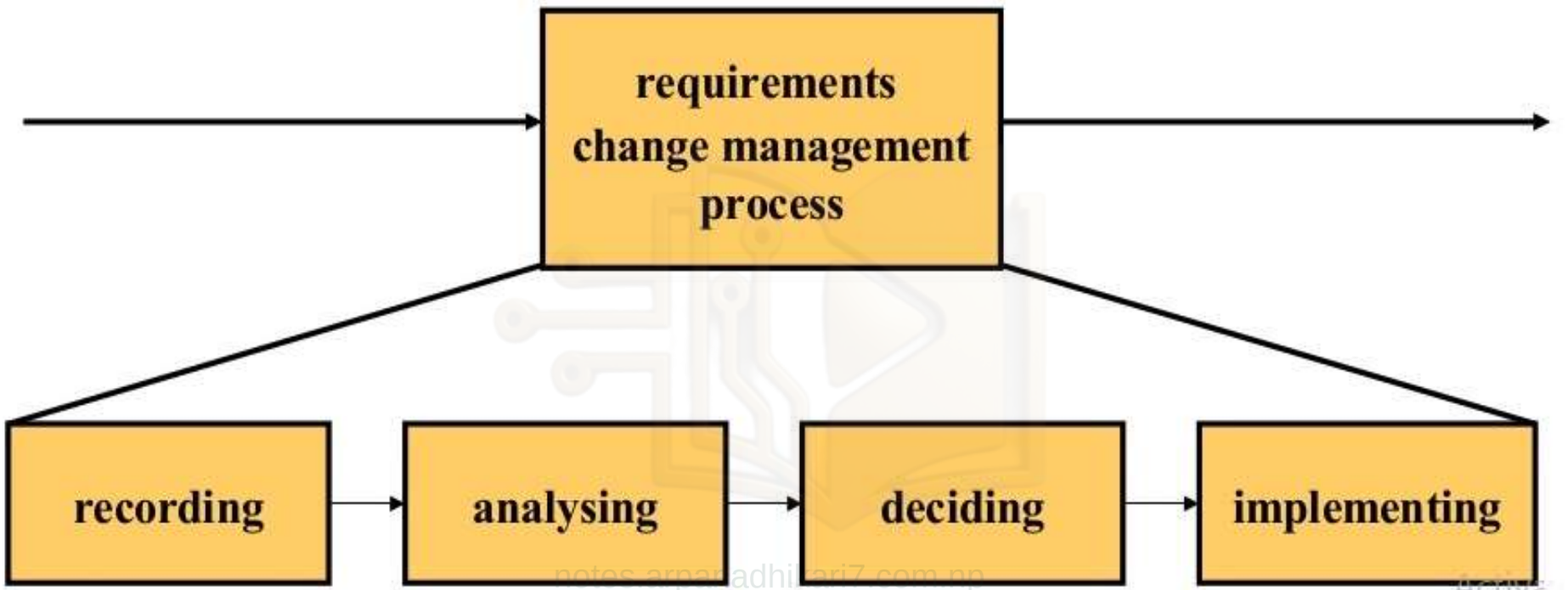
Requirements Change - Why requirements change?

3. Immature Technology
   - Use of immature technology during software development also results in requirements change.
4. Need Changes
   - Sometimes the problem that product is going to address gets changed and the basic needs of our project might change.
   - If so happens, the whole project needs to be rethought.
5. Missing Stakeholders
   - Key stakeholders were left out at the beginning of the project.
   - Stakeholder represent requirements.
   - Missing stakeholders result in missing requirements.
6. Overly Optimistic budget and schedule
   - Promising something that cannot be delivered with the available resources or in the time allotted.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Requirements Change Management



Available on notes.arpanadhikari7.com.np

Requirements Change Management Process

1. Recording a change proposal: In this process, Stakeholders submit proposals for requirements change using a pre-defined form.
2. Analyzing a change request: The proposal submitted by Stakeholders is analyzed to see:
    - if it is valid. Sometimes stakeholders misunderstand the requirements and suggest unnecessary changes.
    - how many requirements and system components are affected.
    - How much it will cost to make change in both time and money
    - What are the benefits of the change
3. Deciding on a change request: Change requests are considered formally by a group or a person who decide whether or not the change should be accepted.
4. Implementing a change: It involves updating the requirements document and informing concerned parties about the update.

Available on notes.arpanadhikari7.com.np

Domain Requirements

- Domain requirements are specific conditions, constraints, or functionalities derived from the application domain that a system must satisfy.
- These requirements are closely tied to the problem space and are often specific to the industry or environment in which the software operates.
- They describe how the software integrates and interacts within its domain.
- Domain requirements can be functional or nonfunctional.
- The basic functions that a system of a specific domain must necessarily exhibit come under this category.
- For instance, in academic software that maintains records of a school or college, the functionality of being able to access the list of faculty and list of students of each grade is a domain requirement.
- These requirements are therefore identified from that domain model and are not user-specific.
- If domain requirements are not satisfied, the system may be unworkable.

Available on notes.arpanadhikari7.com.np

Domain Requirements

some examples of domain requirements in software engineering:

Healthcare: The system must comply with Health Insurance Portability and Accountability Act(HIPAA) regulations for patient data security.

E-commerce: The platform should handle regional tax calculations and currency conversions.

Financial: The software must adhere to Generally Accepted Accounting Principles (GAAP) standards for financial accounting and reporting.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Requirements Modeling Principles

- Principle #1. The information domain of a problem must be represented and understood.
- Principle #2. The functions that the software performs must be defined.
- Principle #3. The behavior of the software (as a consequence of external events) must be represented.
- Principle #4. The models that depict information, function, and behavior must be partitioned in a manner that uncovers detail in a layered (or hierarchical) fashion.
- Principle #5. The analysis task should move from essential information toward implementation detail.

Available on notes.arpanadhikari7.com.np